

PRODUCT DOCUMENTATION

Front-of-Pack (FoP) Rating & Claim Validator

User guide & technical documentation — instant screening of nutrition claims under Codex CAC/GL 23-1997 (with the EU protein variation), disqualifier caution flags, and predicted Nutri-Score 2023 and Health Star Ratings from a single nutrient profile.

Creator	**Dr. Asif Ahmad**
Product	Front-of-Pack (FoP) Rating & Claim Validator — version 1.0
Category	Food Science
Published by	Asif SaaS Studio
Date	21 October 2026
Format	Single-file web application (index.html, 44.4 KB)

Private by design: everything runs on the user's device — no uploads, no accounts, no tracking. Honest by design: every threshold is taken from a named published source, the app shows the value against the threshold for every claim, and it states plainly that its output is guidance, not legal clearance.

Contents

1. Overview — the claims and ratings compliance problem
2. Sources and versions — every table, named
3. Entering a product — the profile, conversions and settings
4. The claim engine — all 24 Codex claims, and the EU protein divergence
5. Caution flags — wording and rationale
6. Nutri-Score 2023 — points, the protein rule, worked examples
7. Health Star Rating — baseline and modifying points, worked example
8. Batch screening — screening a CSV of products
9. Honest scope and limitations
10. How correctness was verified — testing summary
11. Adding the product to the Asif SaaS Studio catalogue (Wix CMS)
12. Product information and credits
13. Appendix — the tested core, verbatim

1. Overview — the claims and ratings compliance problem

A front-of-pack claim is a legal statement, not a marketing flourish. Writing "High Protein", "Source of Fibre" or "Low Sodium" on a pack asserts that the product meets a numerical condition set by a regulator — and the conditions are scattered across documents that were never designed to be read together. Codex Alimentarius guideline CAC/GL 23-1997 sets conditions for some two dozen nutrient content claims, but the basis changes from claim to claim: protein may be judged per 100 g, per 100 ml or per 100 kcal; the European Union judges the same two protein claims by an entirely different route (share of energy); and several claims carry sub-rules, such as the requirement that saturated fat also supply no more than 10% of energy. On top of the claims themselves, two front-of-pack rating systems — Nutri-Score in Europe and the Health Star Rating in Australia and New Zealand — grade the same profile again, each with its own points tables, category rules and edge cases, both recently or periodically revised.

Doing this by hand is slow and error-prone in predictable ways: a threshold checked per 100 g that should have been checked per 100 kcal; a sodium figure read as salt (a factor of 2.5); last year's Nutri-Score table used after the 2023 algorithm replaced it. Each mistake is a regulatory risk discovered, at best, at artwork stage — and at worst, after printing.

This app puts the three screenings on one screen, driven by one nutrient profile entered once:

- **Claim engine** — all 24 Codex nutrient content claims evaluated on every keystroke, each showing PASS or FAIL, the product's value against the exact condition, and for

failures, how far off the product is ("needs to fall by 1.20 g"). A jurisdiction switch re-bases the two protein claims to the EU energy-share rule.

- **Caution flags** — when a product passes claims but is nevertheless high in sodium, sugars, saturated fat or energy, amber cautions say so, because many markets restrict claims on such products even where Codex itself sets no disqualifier.
- **Nutri-Score 2023** — the updated algorithm for general foods and for beverages, with a full auditable points table, the protein-exclusion rule, the red-meat protein cap, and the plain-water rule.
- **Health Star Rating** — all six official categories, baseline and modifying points shown separately, including the rule that decides whether protein points count at all.
- **Batch screening** — paste or upload a CSV of products and screen them all at once; click any row to load it into the main form.

> **Guidance, not legal clearance.** The app computes from the published tables named in Chapter 2. National implementations vary, artwork and wording rules sit outside any nutrient table, and final claim legality needs regulatory review. Chapter 9 states the scope in full — including the two Nutri-Score special categories (cheese; added fats, nuts & seeds) that v1.0 does not implement.

2. Sources and versions — every table, named

Every threshold in the app traces to a row of this table. The versions matter: the Nutri-Score algorithm changed materially in 2023, and the app implements the 2023 text throughout, saying so wherever the older (2017) rule differed.

Engine	Source	Version / detail
Claims	Codex Alimentarius **CAC/GL 23-1997**, Guidelines for Use of Nutrition and Health Claims, Table of conditions for nutrient content claims	Conditions exactly as specified for low / free / source / high claims. NRVs used: protein 50 g, calcium 800 mg, iron 14 mg, vitamin C 100 mg, vitamin D 5 µg
EU protein claims	Regulation (EC) **No 1924/2006**, Annex	"Source of protein" ≥ 12% of energy from protein; "High protein" ≥ 20% of energy
Nutri-Score foods	Santé publique France, Nutri-Score Questions & Answers, version **10 July 2023**, Tables 5 (N) & 6 (P); Scientific Committee update report **2022**, §1.4	N: energy kJ thresholds 335…3350 (max 10 pts), sugars 3.4…51 (max 15 pts), saturated fat 1…10, salt 0.2…4.0 g (max 20 pts). P: protein 2.4…17 (max 7 pts), fibre 3.0…7.4 (max 5 pts), FVL ≤40 / >40 / >60 / >80 = 0/1/2/5. Bands: A ≤ 0, B 1–2, C 3–10, D 11–18, E ≥ 19
Nutri-Score computation rule	SPF Q&A (July 2023), "Calculation of the nutritional score — general case"	N < 11 → score = N − P; **N ≥ 11 → protein points are not counted** (score = N − fibre − FVL points). The 2017 algorithm's extra FVL-points condition on this rule was dropped in 2023 — the app follows the 2023 text (see Chapter 6)
Nutri-Score beverages	SPF Q&A Tables 9 (N) & 10 (P); Scientific Committee beverages update report, **31 Jan 2023**, §6.7.3	Energy 30…390 kJ (non-linear steps), sugars 0.5/2/3.5/5/6/7/8/9/10/11 g, saturated fat 1…10 g, salt as foods, non-nutritive sweeteners present = +4 N. Protein 1.2…3.0 g (max 7 pts, linear), fibre as foods, FVL max **6** points (0/2/4/6). Score = N − P (protein always counted; cap removed). Bands: water = A by rule, B ≤ 2, C 3–6, D 7–9, E ≥ 10
Health Star Rating	Guide for industry to the Health Star Rating Calculator (healthstarrating.gov.au), tables as in the official **Summarised NPSC and HSR Scoring Tables** (2015 update)	Baseline Table 1 (Categories 1, 1D, 2, 2D) and Table 2 (Categories 3, 3D — sugars capped at 10 pts, sodium extended in 90 mg steps); FVNL Table 3 (both columns); protein/fibre Table 4; final-score → stars Table 5 for **all six categories**

Where a published table and common practice diverge at a single boundary, the app documents its choice in-line (see the HSR Table 4 note in Chapter 9). No threshold in the app is an estimate, a rounding of convenience, or drawn from a secondary summary when the primary table was available.

3. Entering a product — the profile, conversions and settings

3.1 Product and settings

Setting	What it drives
Name	Labels the summary, the CSV export and batch rows
Solid (per 100 g) / Beverage (per 100 ml)	Claim limits, Nutri-Score table set, and the default HSR category in batch mode
Serving size (optional)	Display only in the summary — per-serving claim routes are not evaluated (Chapter 9)
FVNL % — fruit, vegetables, nuts & legumes	Nutri-Score FVL points and HSR V points; the "concentrated" toggle selects the HSR concentrated column of Table 3
Red meat toggle	Nutri-Score 2023: protein points for red-meat products are capped at 2
Contains added sugars	The "No added sugars" claim; if natural sugars are present, the app adds the required "contains naturally occurring sugars" statement note
Non-nutritive sweeteners	Nutri-Score beverages: +4 N points when present
Plain water	Nutri-Score class A by rule, regardless of the tables
Jurisdiction: Codex / EU	Re-bases the two protein claims (Chapter 4.3); all other claims are unchanged
HSR category	One of the six official categories: 1 (non-dairy beverages), 1D (dairy beverages), 2 (general foods), 2D (dairy foods), 3 (oils & spreads), 3D (cheese > 320 mg Ca/100 g)

3.2 The nutrient profile, and the two conversions that cause the most errors

Enter per 100 g (solids) or per 100 ml (beverages): energy, protein, fibre, total sugars, total fat, saturated fat, sodium, calcium, iron, vitamin C and vitamin D. Two pairs of fields are linked, because each pair causes a classic labelling error:

- **Energy in kcal and kJ** — the fields are linked at **1 kcal = 4.184 kJ**; edit either and the other follows. Nutri-Score and HSR points tables are defined in kJ; Codex energy claims are judged in kcal; the app converts once, centrally, so the two engines can never disagree about a product's energy.

- **Sodium (mg) and salt (g)** — linked at **sodium mg = salt g × 400** (equivalently, salt = sodium × 2.5). Codex sodium claims are judged in mg of sodium; the Nutri-Score salt row is in grams of salt. Entering a salt figure into a sodium field (or the reverse) is the single most common cause of a wrong claim verdict — the linked fields remove it.

Validation is live and specific: negative values, any nutrient above 100 g per 100 g, FVNL outside 0–100, and saturated fat exceeding total fat each raise an inline message naming the problem, rather than a silently wrong result.

4. The claim engine — all 24 Codex claims, and the EU protein divergence

Every condition below is applied exactly as written in CAC/GL 23-1997, on the basis shown. Values are per 100 g for solids and per 100 ml for beverages unless a per-100-kcal route is stated. A claim at exactly the threshold **passes** (conditions are "not more than" / "not less than"); the test suite verifies every boundary at the threshold and at a hair above or below it.

#	Claim	Condition (Codex)	Notes / route used
1	Low energy	≤ 40 kcal (solids); ≤ 20 kcal (beverages)	Judged on energy per 100 g / 100 ml
2	Energy-free	≤ 4 kcal per 100 ml	Codex sets this claim for liquids; shown as not applicable for solids
3	Low fat	≤ 3 g (solids); ≤ 1.5 g (beverages)	
4	Fat-free	≤ 0.5 g	Both forms
5	Low saturated fat	≤ 1.5 g (solids) / ≤ 0.75 g (beverages) **AND** saturated fat ≤ 10% of total energy	Two-part condition — failing either part fails the claim, and the app says which part failed
6	Saturated-fat-free	≤ 0.1 g	Some national implementations add a trans-fat condition (Chapter 9)
7	Low sugars	≤ 5 g (solids); ≤ 2.5 g (beverages)	
8	Sugars-free	≤ 0.5 g	Both forms
9	No added sugars	No added mono-/disaccharides or other food used for its sweetening properties	Driven by the "contains added sugars" setting; if the product passes but contains natural sugars, the app notes the pack must also state "contains naturally occurring sugars"
10	Low sodium	≤ 120 mg sodium	Judged on sodium, not salt (Chapter 3.2)
11	Very low sodium	≤ 40 mg sodium	
12	Sodium-free	≤ 5 mg sodium	
13	Source of protein	≥ 5 g per 100 g (10% NRV) **or** ≥ 2.5 g per 100 kcal (5% NRV); beverages: ≥ 2.5 g per 100 ml or the per-100-kcal route	The app names the route met. EU basis replaces both routes — see 4.3
14	High protein	≥ 10 g per 100 g (20% NRV) **or** ≥ 5 g per 100 kcal (10% NRV); beverages: ≥ 5 g per 100 ml or the per-100-kcal route	As above
15	Source of fibre		Any one route suffices; the app names the route(s) met

		≥ 3 g per 100 g, **or** ≥ 1.5 g per 100 ml, **or** ≥ 1.5 g per 100 kcal	
16	High fibre	≥ 6 g per 100 g, **or** ≥ 3 g per 100 ml, **or** ≥ 3 g per 100 kcal	As above
17	Source of calcium	≥ 15% of NRV (800 mg) per 100 g = 120 mg; ≥ 7.5% per 100 ml = 60 mg	NRV route
18	High calcium	≥ 30% of NRV per 100 g = 240 mg; ≥ 15% per 100 ml = 120 mg	Twice the "source" level
19	Source of iron	≥ 15% of NRV (14 mg) per 100 g = 2.1 mg; ≥ 7.5% per 100 ml	NRV route
20	High iron	≥ 30% of NRV per 100 g = 4.2 mg; ≥ 15% per 100 ml	
21	Source of vitamin C	≥ 15% of NRV (100 mg) per 100 g = 15 mg; ≥ 7.5% per 100 ml	NRV route
22	High vitamin C	≥ 30% of NRV per 100 g = 30 mg; ≥ 15% per 100 ml	
23	Source of vitamin D	≥ 15% of NRV (5 µg) per 100 g = 0.75 µg; ≥ 7.5% per 100 ml	NRV route
24	High vitamin D	≥ 30% of NRV per 100 g = 1.5 µg; ≥ 15% per 100 ml	

4.1 How results are shown

Each claim row shows the claim, PASS / FAIL / not-applicable, the product's own value, the condition in words, and — on failure — the size of the gap ("needs to fall by 1.20 g", "needs 0.45 mg more"), so a formulator can see at a glance whether a claim is one reformulation step away or out of reach. A header line totals the passes (e.g. "6 of 24 claims passed").

4.2 The protein routes, in detail

Protein is where claim-checking most often goes wrong, because Codex offers two independent routes and either suffices: an absolute route (per 100 g or 100 ml, tied to the 50 g NRV) and an energy-density route (per 100 kcal). A lean, watery product can fail the absolute route and pass on density; a dense product can do the reverse. The app evaluates both on every product and names the route used — never just the verdict.

4.3 The EU protein divergence

Regulation (EC) No 1924/2006 judges the same two claims by a different quantity altogether: the share of the product's energy supplied by protein — **≥ 12%** for "source of protein", **≥ 20%** for "high protein". Switching the jurisdiction selector to EU replaces the Codex routes for claims 13–14 only; the other 22 claims are identical. The two systems genuinely disagree, in both directions:

> **Constructed example 1 (EU stricter at the top).** Protein 10 g per 100 g, energy 267 kcal per 100 g. Protein supplies 10 × 4 / 267 = **15.0% of energy**. Codex: "High protein" **passes** on the per-100 g route (10 g ≥ 10 g). EU: "Source of protein" passes (15% ≥ 12%) but "High protein" **fails** (15% < 20%).

> **Constructed example 2 (EU stricter at the bottom).** Protein 5.5 g per 100 g, energy 200 kcal per 100 g. Protein supplies 5.5 × 4 / 200 = **11.0% of energy**. Codex: "Source of protein" **passes** per 100 g (5.5 g ≥ 5 g). EU: "Source of protein" **fails** (11% < 12%). A product formulated to the Codex line and sold into the EU on that basis would carry an unlawful claim — exactly the class of error the jurisdiction switch exists to catch.

5. Caution flags — wording and rationale

Codex GL 23 sets conditions for claims; it does **not**, in general, disqualify a product from all claims because it is high in some other nutrient. Many national regimes, retailer standards and front-of-pack schemes are less forgiving — claims on products that are simultaneously high in sodium, sugars, saturated fat or energy are restricted, challenged or reputationally costly in a long list of markets. The app therefore separates the two ideas strictly:

- The claim verdicts in Chapter 4 are **Codex verdicts** — nothing else is allowed to change them.
- Separately, if **at least one claim passes**, the app raises **amber cautions** (never red "disqualifications") when the profile is high in: sodium (≥ 500 mg per 100 g/ml), sugars (≥ 15 g per 100 g, solids), saturated fat (≥ 5 g per 100 g/ml), or energy (≥ 400 kcal per 100 g/ml). Each caution names the value and the threshold crossed, in the form "sugars are 18 g per 100 g (≥15)".

The wording is deliberate: a caution says "many markets restrict claims on such products" — it does not say the claim is prohibited, because under the app's stated source it is not. Treating a caution as a Codex rule would be its own kind of mislabelling; the app refuses to blur the line in either direction.

6. Nutri-Score 2023 — points, the protein rule, worked examples

6.1 General foods — the points

Negative (N) points accumulate from four components; positive (P) points from three. Each component scores by thresholds stepped through the tables of Chapter 2:

Component	Points range	Scale (summary)
Energy (N)	0–10	Thresholds every 335 kJ from 335 to 3350 kJ per 100 g
Sugars (N)	0–15	Thresholds from 3.4 g to 51 g per 100 g (2023 scale)
Saturated fat (N)	0–10	1 g steps from 1 to 10 g per 100 g
Salt (N)	0–20	0.2 g steps from 0.2 to 4.0 g per 100 g
Protein (P)	0–7	Thresholds 2.4…17 g per 100 g; **capped at 2 for red-meat products** (2023 rule)
Fibre (P)	0–5	Thresholds 3.0…7.4 g per 100 g
Fruit / vegetables / legumes (P)	0 / 1 / 2 / 5	≤ 40% → 0; > 40% → 1; > 60% → 2; > 80% → 5

6.2 The computation rule — and why the app implements it this way

For general foods the 2023 algorithm is:

N < 11 → final score = N − P · N ≥ 11 → final score = N − fibre points − FVL points (protein points NOT counted)

The protein exclusion is the most-misapplied rule in the system, because the 2017 algorithm attached a further condition to it (the exclusion bit only when FVL points were low). The July 2023 Questions & Answers text states the general case without that condition: once N reaches 11, protein points are simply not counted. The app implements the **2023 text as published**, and says so in its own rule line under every result, because a tool that silently kept the friendlier 2017 condition would systematically over-grade exactly the products — energy-dense, salty, fatty — that the 2023 update set out to grade harder. The test suite includes the discriminating case: a product at N = 11 with maximum FVL points still has its protein excluded (score 4 → C, where the 2017 form would have counted the protein).

Grades: **A** ≤ 0, **B** 1–2, **C** 3–10, **D** 11–18, **E** ≥ 19. The app shows all five grade chips with the earned one highlighted, the final score with its N and P totals, and an expandable table giving every component's value and points — the full audit trail, not just the letter.

6.3 Beverages

Beverages use their own tables (Chapter 2): energy in non-linear kJ steps, sugars from 0.5 g, saturated fat and salt as for foods, **+4 N points if non-nutritive sweeteners are present**, protein points on a lower linear scale (1.2…3.0 g, max 7), fibre as for foods, and FVL worth up to **6** points (0/2/4/6). The computation is always final = N − P — the 2023 beverage update removed the protein cap and applies no exclusion. Bands: plain water is **A by rule**; otherwise **B** ≤ 2, **C** 3–6, **D** 7–9, **E** ≥ 10.

6.4 Worked examples — published classifications, reproduced

Product (per 100 g / 100 ml)	Profile used	N	Final score	Grade
Chocolate-hazelnut spread (Nutella-type; published 2023 grade E)	2252 kJ, sugars 56.3 g, sat. fat 10.6 g, sodium 43 mg, protein 6.3 g, fibre 3.4 g	31	30	**E** ✓
Semi-skimmed milk (SPF Q&A: "largely classified as B")	195 kJ, sugars 4.8 g, sat. fat 1.1 g, sodium 44 mg, protein 3.4 g	—	0	**B** ✓
Whole milk (SPF Q&A: "classified as C")	265 kJ, sugars 4.7 g, sat. fat 2.4 g, sodium 44 mg, protein 3.3 g	—	3	**C** ✓
100% orange juice (2023 beverages report outcome C)	188 kJ, sugars 8.8 g, sat. fat 0.1 g, sodium 4 mg, protein 0.7 g, FVL 100%	—	4	**C** ✓
Cola (sugar-sweetened soda)	180 kJ, sugars 10.6 g, sodium 4 mg	—	12	**E** ✓

Nutrient values are from standard product labels; expected grades are the officially published classifications, not the app's own output read back (Chapter 10 describes the verification method).

7. Health Star Rating — baseline and modifying points, worked example

The Health Star Rating (Australia / New Zealand) grades from 0.5 to 5 stars in half-star steps. The computation is: **final score = baseline points − V points − P points − F points**, and the final score is converted to stars by a table that depends on the product's category. The app implements **all six categories**: 1 (non-dairy beverages), 1D (dairy beverages), 2 (general foods), 2D (dairy foods), 3 (oils & spreads) and 3D (cheese with > 320 mg calcium per 100 g).

7.1 Baseline points (the "bad" points)

Four components, each stepped from the official tables. Categories 1, 1D, 2 and 2D use baseline **Table 1**; categories 3 and 3D use **Table 2**, which differs in three ways that matter:

sugars points are **capped at 10**, the sodium scale is extended beyond 900 mg in uniform 90 mg steps (to 2700 mg), and the saturated-fat scale runs in 1 g steps to 30 g.

Component	Table 1 (Cats 1, 1D, 2, 2D)	Table 2 (Cats 3, 3D)
Energy	Thresholds every 335 kJ from 335 to 3685 kJ (0–11 pts) — both tables	
Saturated fat	Log-style scale 1…90 g (0–30 pts)	1 g steps, 1…30 g (0–30 pts)
Total sugars	Thresholds 5…99 g (0–22 pts)	Thresholds 5…45 g (**max 10 pts**)
Sodium	Scale 90…8106 mg (0–30 pts)	90 mg steps, 90…2700 mg (0–30 pts)

7.2 Modifying points (the "good" points)

- **V points** — from the FVNL percentage, Table 3, which has two columns: the ordinary column (0–7 points; a product that is essentially all FVNL, ≥ 99.5%, earns an 8th point) and the **concentrated** column for concentrated fruit/vegetable ingredients (selected by the app's "concentrated" toggle).
- **P points** — protein, thresholds 1.6…50 g (0–15 pts), Table 4.
- **F points** — fibre, thresholds 0.9…20 g (0–15 pts), Table 4. Fibre points do not apply in the beverage categories 1 and 1D.

7.3 The protein rule (baseline ≥ 13)

Protein points are **not** automatic. They count only if the product's baseline score is **below 13**, or the product earns **5 or more V points**:

$$\text{protein counts} \iff \text{baseline} < 13 \;\; \text{OR} \;\; V \geq 5$$

The rule exists so that a fundamentally unhealthy product cannot buy back its rating with added protein. The app shows the rule's outcome explicitly in every result ("protein counted" / "protein NOT counted (baseline ≥ 13 and V points < 5)"), and the test suite verifies both branches at a baseline of exactly 14: excluded with V < 5 (score 14 → 2 stars), counted with V = 5 (score 3 → 3 stars).

7.4 Worked example — the official Guide's soft drink

The worked example in the official Guide for industry is a sweetened soft drink in Category 1: 204 kJ, protein 0.2 g, saturated fat 0 g, sugars 11.5 g, fibre 0 g, sodium 15 mg per 100 ml. Baseline: energy 0 (204 ≤ 335) + saturated fat 0 + sugars 2 (11.5 g passes the 5 g and 9 g thresholds) + sodium 0 = **2**. Modifying points: V 0, P 0, F not applicable (Category 1). Final score **2** → in Category 1's Table 5 row, a final score of 2 is **1 star**. The app reproduces baseline 2, final 2, 1 star exactly. (The same final score in Category 1D would be 3 stars and in Category 2, 3.5 stars — the category choice is part of the rating, which is why the app never hides it.)

8. Batch screening — screening a CSV of products

For screening a product range at once, the batch box accepts pasted CSV text or an uploaded `.csv` / `.txt` file (read in-page; nothing leaves the browser). Columns, in the positional fallback order:

> name, form, energy_kcal, protein, fibre, sugars, fat, satfat, sodium_mg, fvn_pct, calcium, iron, vitc, vitd

- Separators may be commas, semicolons or tabs. A header row is matched **by column name** (case- and separator-insensitive), so columns may be reordered; without a recognisable header, the positional order above is assumed.
- `form` is read as a beverage if it contains "bev", "drink" or "liquid"; anything else is a solid. Batch defaults: HSR category 1 for beverages, 2 for solids; the added-sugars / sweetener / red-meat flags are off — load a row into the main form (click it) to adjust flags for that product.
- Energy is entered in kcal (converted to kJ internally); micronutrient columns may be left blank (read as 0).
- A row with a non-numeric value in a nutrient column is **reported by line number** ("Line 3: non-numeric value in a nutrient column — row skipped") and skipped — never silently dropped, never guessed at.

```
name,form,energy_kcal,protein,fibre,sugars,fat,satfat,sodium_mg,fvn_pct,calcium,iron,vitc,vitd
Oat bar,solid,410,9,7,18,14,3,180,12,40,2.6,0,0
```

Results show, per product: claims passed (of 24), the Nutri-Score grade and the HSR stars. Outputs for the single-product form are a copyable text summary and a full CSV download (ratings, every claim with status, value and requirement, and the caution flags).

9. Honest scope and limitations

- **Guidance, not legal clearance.** Predictions are computed from the published tables named in Chapter 2. National implementations vary — for example, some markets add a trans-fat condition to "saturated-fat-free", and claim wording, language, prominence and accompanying-statement rules sit outside the nutrient tables entirely. Final claim legality for a specific market needs regulatory review.
- **Profile in, not recipe in.** The app takes a nutrient profile (per 100 g / 100 ml) as its input; it does not compute a profile from a recipe's ingredients. Ingredient-level formulation and costing are the job of the companion LP Recipe & Cost Optimizer; a recipe-to-profile feed is a staged v1.1 item, not something v1.0 pretends to do.
- **Two Nutri-Score special categories are NOT implemented.** The 2023 algorithm's special **cheese** rule and its **added fats / nuts & seeds** category are

outside v1.0, which covers general foods and beverages. Classifying a cheese or an oil with the general tables will mis-grade it; the in-app scope note says the same.

- **Classification is the user's.** Whether a product is a solid or a beverage, and which of the six HSR categories it belongs to, materially change its ratings (Chapter 7.4). The app takes these as inputs and displays them with every result; it does not second-guess them.
- **Ratings by rule.** Plain water is Nutri-Score A by rule (the app's water checkbox covers this). In the official HSR system, water and fresh/minimally processed produce are likewise automatic 5-star products — enter Category 1 for water and treat the table result accordingly.
- **HSR Table 4 boundary note.** In the official summarised Table 4, the protein step at 3.2 g is printed "≥ 3.2" while every other step uses ">". The app applies ">" uniformly, so a product at exactly 3.2 g protein may differ by one P point from a strict reading of that single printed cell.
- **Per-serving claim routes are not evaluated.** Codex also permits certain claims on a per-serving basis; the app's serving-size field is display-only. Products passing only via a serving route will show FAIL here — the conservative direction for a screening tool.
- **One FVNL input serves two definitions.** The Nutri-Score FVL definition and the HSR FVNL definition differ slightly at the edges (nuts and certain oils, for instance). Enter the figure appropriate to your product; the app uses it for both engines and says so.
- **Privacy.** No backend is needed or faked: no server, no storage, no API keys — profiles you paste never leave the browser.

10. How correctness was verified — testing summary

The app's PURE CORE (conversions, claim engine, both rating algorithms, batch parser) was extracted from the page and run in Node against expectations derived **independently, by hand, from the published tables of Chapter 2** — every boundary value computed before the harness ran, and the worked examples taken from officially published classifications rather than from the app's own output. The full inline scripts (core + DOM) were extracted and syntax-checked. **Result: 110 / 110 checks passed.**

Group	Checks	Result
Salt/sodium conversions (1.0 g salt = 400 mg sodium; 120 mg = 0.3 g salt)	3	✓
Claim boundaries — exact-at-threshold PASS and +ε FAIL (low fat, fat-free, low / very low / sodium-free, low energy solid & beverage, energy-free, low sugars, sugars-free, saturated-fat-free)	26	✓
Saturated-fat 10%-of-energy sub-rule flips (1.5 g @ 100 kcal FAIL vs @ 200 kcal PASS; beverage case)	3	✓
Protein / fibre routes, including the per-100-kcal route (route named in output)	9	✓
Micronutrient NRV boundaries (Ca 120/119 & 240, Fe 2.1, vitamin C 15, vitamin D 0.75, beverage Ca 60)	7	✓
No-added-sugars logic + naturally-occurring-sugars statement note	3	✓
Claim inventory — 24 claims evaluated in both jurisdictions	2	✓
EU/Codex divergence (15% protein-energy: EU source PASS / high FAIL, Codex high PASS; 11%: EU source FAIL, Codex source PASS)	5	✓
Caution flags (raised on sodium 600 mg; none on a clean profile)	2	✓
Nutri-Score foods — published example + rule cases (protein exclusion at N = 11 with maximum FVL; red-meat cap 7 → 2; 335/336 kJ boundary)	15	✓
Nutri-Score beverages — official classifications (milk B, whole milk C, juice C, cola E; sweetener +4 flips B → C; water → A)	10	✓
Health Star Rating — official worked example + rules (protein rule both branches; category effect Cat 1 vs 1D; Table 2 sugars cap; concentrated-FVNL column; fibre n/a in beverage categories)	17	✓
Batch parser (3-product batch; header / semicolon / no-header forms; bad-line numbering; empty input)	12	✓

One honest note on the process: during development, **eight initial test expectations** were themselves wrong — hand-derivation slips made when the expectations were first written down — and were corrected after re-derivation from the published tables; in each case the engine's behaviour was the correct one. They are recorded here because a test suite that has never caught its own author being wrong is not evidence of much; this one was checked in both directions.

11. Adding the product to the Asif SaaS Studio catalogue (Wix CMS)

Step 1. In the Wix dashboard, open the CMS collection **SaasProducts**.

Step 2. Edit (or add) the item with slug `fop-claim-validator`.

Step 3. Paste the full contents of `index.html` into the **appHtml** field and set the listing fields:

Field	Value
Title	Front-of-Pack (FoP) Rating & Claim Validator
Description	Screen nutrition claims under Codex (with the EU protein variation) and predict Nutri-Score 2023 and Health Star Ratings from a nutrient profile.
Category	Food Science
Slug	fop-claim-validator
Version	1.0
Published	On

Step 4. Save, open the live app page, and run the smoke test with the app's default product (Wholegrain muesli bar, already filled in): expect **Nutri-Score C** (score 10, N 13 − P 3), **Health Star Rating 3.5 stars** (final score 0, baseline 11), **6 of 24 claims passed** (No added sugars; Low sodium; Source of protein; Source of fibre; High fibre; Source of iron) and **two caution flags** (sugars 18 g ≥ 15; energy 410 kcal ≥ 400). Then paste the sample line from the batch box placeholder and click "Screen batch": the Oat bar row should show **6 / 24** claims, Nutri-Score **C**, HSR **3.5 ★**.

12. Product information and credits

Item	Detail
Product	Front-of-Pack (FoP) Rating & Claim Validator, version 1.0
Creator	Dr. Asif Ahmad
Published by	Asif SaaS Studio
Category	Food Science
Format	Single-file web application (index.html, 44,421 bytes); all CSS and JavaScript inline; no build step, no server, no storage, no external requests
Licence	MIT
Sources	Codex CAC/GL 23-1997; Regulation (EC) No 1924/2006 (Annex); Santé publique France Nutri-Score Q&A, 10 July 2023, and Scientific Committee update reports 2022 / Jan 2023; HSR Guide for industry to the Health Star Rating Calculator (summarised scoring tables, 2015 update). Nutri-Score is a programme of Santé publique France; the Health Star Rating system is an Australian/New Zealand government initiative — this app is an independent screening tool, not an official calculator of either system

Appendix — the tested core, verbatim

The complete PURE CORE of the shipped app — conversions, the Codex claim engine, the Nutri-Score 2023 algorithm, the Health Star Rating algorithm and the batch CSV parser — reproduced exactly as tested (110 / 110 checks). This is the code the documentation above describes.

```
'use strict';
/* ============ FoP Validator — PURE CORE (no DOM) ============
   Sources:
   - Claims: Codex CAC/GL 23-1997 (Guidelines for Use of Nutrition and Health Claims);
     EU protein variation: Regulation (EC) No 1924/2006, Annex.
   - Nutri-Score: 2023 updated algorithm — Santé publique France, "Nutri-Score
     Questions & Answers" v. 10 July 2023 (Tables 5, 6, 9, 10); Scientific Committee
     update reports 2022 (main algorithm, §1.4 thresholds) and Jan 2023 (beverages, §6.7.3).
   - Health Star Rating: "Guide for industry to the Health Star Rating Calculator"
     tables (baseline Tables 1-2, FVNL Table 3, protein/fibre Table 4, final-score
     Table 5) as summarised in the official summarised scoring tables (2015 update).
   ============================================================ */
var KJ_PER_KCAL = 4.184;
var NRV = { protein: 50, calcium: 800, iron: 14, vitC: 100, vitD: 5 };

function ptsAbove(value, thresholds){ var n=0; for (var i=0;i<thresholds.length;i++){ if (value > thresholds[i]
+ 1e-9) n++; } return n; }
function saltGFromSodiumMg(mg){ return mg/400; }
function sodiumMgFromSaltG(g){ return g*400; }
function fmt(x, d){ var n = Math.round(x*Math.pow(10,d))/Math.pow(10,d); return (''+n); }

/* ---------- Nutri-Score 2023 tables ---------- */
var NS = {
  foodEnergy: [335,670,1005,1340,1675,2010,2345,2680,3015,3350],
  foodSugars: [3.4,6.8,10,14,17,20,24,27,31,34,37,41,44,48,51],
```

```
  satFat:     [1,2,3,4,5,6,7,8,9,10],
  salt:       [0.2,0.4,0.6,0.8,1.0,1.2,1.4,1.6,1.8,2.0,2.2,2.4,2.6,2.8,3.0,3.2,3.4,3.6,3.8,4.0],
  foodProtein:[2.4,4.8,7.2,9.6,12,14,17],
  fibre:      [3.0,4.1,5.2,6.3,7.4],
  bevEnergy:  [30,90,150,210,240,270,300,330,360,390],
  bevSugars:  [0.5,2,3.5,5,6,7,8,9,10,11],
  bevProtein: [1.2,1.5,1.8,2.1,2.4,2.7,3.0]
};
function nsFvlFood(pct){ return pct>80?5:(pct>60?2:(pct>40?1:0)); }
function nsFvlBev(pct){ return pct>80?6:(pct>60?4:(pct>40?2:0)); }

function nutriScore(p){
  var saltG = saltGFromSodiumMg(p.sodiumMg||0);
  var rows = [], N = 0, P = 0, proteinCounted = true, rule = '';
  if (p.form === 'beverage'){
    var eE = ptsAbove(p.energyKj, NS.bevEnergy), eS = ptsAbove(p.sugars, NS.bevSugars),
        eF = ptsAbove(p.satfat, NS.satFat), eNa = ptsAbove(saltG, NS.salt),
        eSw = p.sweeteners ? 4 : 0;
    N = eE+eS+eF+eNa+eSw;
    var bPr = ptsAbove(p.protein, NS.bevProtein), bFi = ptsAbove(p.fibre, NS.fibre), bFv = nsFvlBev(p.fvn||0);
    P = bPr+bFi+bFv;
    rows = [
      ['Energy', fmt(p.energyKj,0)+' kJ/100 ml', eE], ['Sugars', fmt(p.sugars,2)+' g/100 ml', eS],
      ['Saturated fat', fmt(p.satfat,2)+' g/100 ml', eF], ['Salt', fmt(saltG,2)+' g/100 ml', eNa],
      ['Non-nutritive sweeteners', p.sweeteners?'present':'absent', eSw],
      ['Protein', fmt(p.protein,2)+' g/100 ml', bPr], ['Fibre', fmt(p.fibre,2)+' g/100 ml', bFi],
      ['Fruit/veg/legumes', fmt(p.fvn||0,0)+' %', bFv]
    ];
    var bScore = N - P, bGrade;
    if (p.water) bGrade = 'A';
    else bGrade = bScore<=2?'B':(bScore<=6?'C':(bScore<=9?'D':'E'));
    rule = p.water ? 'Plain water is class A by rule (2023 beverages report).' :
      'Beverages: final score = N − P (no protein-exclusion rule; protein cap removed in the 2023 beverage
update). Bands: B ≤2, C 3–6, D 7–9, E ≥10; only water is A.';
    return { N:N, P:P, proteinPts:bPr, fibrePts:bFi, fvlPts:bFv, proteinCounted:true, score:bScore,
grade:bGrade, rows:rows, rule:rule };
  }
  var fE = ptsAbove(p.energyKj, NS.foodEnergy), fS = ptsAbove(p.sugars, NS.foodSugars),
      fF = ptsAbove(p.satfat, NS.satFat), fNa = ptsAbove(saltG, NS.salt);
  N = fE+fS+fF+fNa;
  var rawPr = ptsAbove(p.protein, NS.foodProtein);
  var fPr = p.redMeat ? Math.min(rawPr, 2) : rawPr;
  var fFi = ptsAbove(p.fibre, NS.fibre), fFv = nsFvlFood(p.fvn||0);
  var score;
  if (N >= 11){ proteinCounted = false; score = N - fFi - fFv; P = fFi + fFv; }
  else { score = N - (fPr+fFi+fFv); P = fPr+fFi+fFv; }
  var grade = score<=0?'A':(score<=2?'B':(score<=10?'C':(score<=18?'D':'E')));
  rows = [
    ['Energy', fmt(p.energyKj,0)+' kJ/100 g', fE], ['Sugars', fmt(p.sugars,2)+' g/100 g', fS],
    ['Saturated fat', fmt(p.satfat,2)+' g/100 g', fF], ['Salt', fmt(saltG,2)+' g/100 g', fNa],
    ['Protein'+(p.redMeat?' (red-meat cap 2)':''), fmt(p.protein,2)+' g/100 g', fPr],
    ['Fibre', fmt(p.fibre,2)+' g/100 g', fFi], ['Fruit/veg/legumes', fmt(p.fvn||0,0)+' %', fFv]
  ];
  rule = N>=11 ? 'N ≥ 11: under the 2023 algorithm protein points are NOT counted (final = N − fibre − fruit/
veg/legumes points). Bands: A ≤0, B 1–2, C 3–10, D 11–18, E ≥19.'
               : 'N < 11: final score = N − P (all positive points counted). Bands: A ≤0, B 1–2, C 3–10, D 11–
18, E ≥19.';
  return { N:N, P:P, proteinPts:fPr, rawProteinPts:rawPr, fibrePts:fFi, fvlPts:fFv,
proteinCounted:proteinCounted, score:score, grade:grade, rows:rows, rule:rule };
}

/* ---------- Health Star Rating tables ---------- */
var HSR_ENERGY = [335,670,1005,1340,1675,2010,2345,2680,3015,3350,3685];
var HSR_SATFAT_A =
[1,2,3,4,5,6,7,8,9,10,11.2,12.5,13.9,15.5,17.3,19.3,21.6,24.1,26.9,30,33.5,37.4,41.7,46.6,52,58,64.7,72.3,80.6,9
0];
var HSR_SATFAT_B = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30];
var HSR_SUGARS_A = [5,9,13.5,18,22.5,27,31,36,40,45,49,54,58,63,67,72,76,81,85,90,94,99];
var HSR_SUGARS_B = [5,9,13.5,18,22.5,27,31,36,40,45];
var HSR_SODIUM_A =
[90,180,270,360,450,540,630,720,810,900,1005,1121,1251,1397,1559,1740,1942,2168,2420,2701,3015,3365,3756,4192,46
79,5223,5829,6506,7262,8106];
var HSR_SODIUM_B =
[90,180,270,360,450,540,630,720,810,900,990,1080,1170,1260,1350,1440,1530,1620,1710,1800,1890,1980,2070,2160,225
0,2340,2430,2520,2610,2700];
var HSR_PROTEIN = [1.6,3.2,4.8,6.4,8,9.6,11.6,13.9,16.7,20,24,28.9,34.7,41.6,50];
var HSR_FIBRE = [0.9,1.9,2.8,3.7,4.7,5.4,6.3,7.3,8.4,9.7,11.2,13,15,17.3,20];
```

```
function hsrVPoints(pct, concentrated){
  if (pct >= 99.5) return 8;
  if (concentrated){
    if (pct>=90) return 7; if (pct>=80) return 6; if (pct>=67) return 5; if (pct>=63) return 4;
    if (pct>=52) return 3; if (pct>=43) return 2; if (pct>=25) return 1; return 0;
  }
  if (pct>95) return 7; if (pct>90) return 6; if (pct>80) return 5; if (pct>75) return 4;
  if (pct>67) return 3; if (pct>60) return 2; if (pct>40) return 1; return 0;
}
function hsrStarsFor(cat, s){
  function r(lo,hi){ return s>=lo && s<=hi; }
  switch(cat){
    case '1':  return s<=-6?5:(s===-5?4.5:(s===-4?4:(s===-3?3.5:(s===-2?3:(s===-1?2.5:(s===0?2:(s===1?1.5:
(s===2?1:0.5)))))))));
    case '1D': case '2D': return s<=-2?5:(s===-1?4.5:(s===0?4:(s===1?3.5:(s===2?3:(s===3?2.5:(s===4?2:(s===5?
1.5:(s===6?1:0.5)))))))));
    case '2':  return s<=-11?5:(r(-10,-7)?4.5:(r(-6,-2)?4:(r(-1,2)?3.5:(r(3,6)?3:(r(7,11)?2.5:(r(12,15)?2:
(r(16,20)?1.5:(r(21,24)?1:0.5)))))))));
    case '3':  return s<=13?5:(r(14,16)?4.5:(r(17,20)?4:(r(21,23)?3.5:(r(24,27)?3:(r(28,30)?2.5:(r(31,34)?2:
(r(35,37)?1.5:(r(38,41)?1:0.5)))))))));
    case '3D': return s<=22?5:(r(23,24)?4.5:(r(25,26)?4:(r(27,28)?3.5:(r(29,30)?3:(r(31,32)?2.5:(r(33,34)?2:
(r(35,36)?1.5:(r(37,38)?1:0.5)))))))));
  }
  return 0;
}
var HSR_CAT_LABEL = { '1':'Category 1 — non-dairy beverages', '1D':'Category 1D — dairy beverages',
'2':'Category 2 — general foods', '2D':'Category 2D — dairy foods', '3':'Category 3 — oils & spreads',
'3D':'Category 3D — cheese (>320 mg Ca/100 g)' };

function healthStars(p){
  var cat = p.hsrCategory || '2';
  var tableB = (cat==='3' || cat==='3D');
  var eE = ptsAbove(p.energyKj, HSR_ENERGY);
  var eF = ptsAbove(p.satfat, tableB?HSR_SATFAT_B:HSR_SATFAT_A);
  var eS = ptsAbove(p.sugars, tableB?HSR_SUGARS_B:HSR_SUGARS_A);
  var eNa = ptsAbove(p.sodiumMg, tableB?HSR_SODIUM_B:HSR_SODIUM_A);
  var baseline = eE+eF+eS+eNa;
  var v = hsrVPoints(p.fvn||0, !!p.fvnConcentrated);
  var pRaw = ptsAbove(p.protein, HSR_PROTEIN);
  var proteinCounts = (baseline < 13) || (v >= 5);
  var pPts = proteinCounts ? pRaw : 0;
  var fibreApplies = !(cat==='1' || cat==='1D');
  var fPts = fibreApplies ? ptsAbove(p.fibre, HSR_FIBRE) : 0;
  var score = baseline - v - pPts - fPts;
  var stars = hsrStarsFor(cat, score);
  var rows = [
    ['Energy (baseline)', fmt(p.energyKj,0)+' kJ', eE], ['Saturated fat (baseline)', fmt(p.satfat,2)+' g', eF],
    ['Total sugars (baseline)', fmt(p.sugars,2)+' g', eS], ['Sodium (baseline)', fmt(p.sodiumMg,0)+' mg', eNa],
    ['FVNL — V points', fmt(p.fvn||0,0)+' %'+(p.fvnConcentrated?' (concentrated)':''), v],
    ['Protein — P points', fmt(p.protein,2)+' g', pPts], ['Fibre — F points', fmt(p.fibre,2)+' g', fPts]
  ];
  var rule = 'Baseline '+baseline+'; protein '+(proteinCounts?'counted':'NOT counted (baseline ≥13 and V points
<5)')+
    '; fibre '+(fibreApplies?'counted':'not applicable to beverage categories 1/1D')+
    '. Final = baseline − V − P − F = '+score+' → '+stars+' stars in '+HSR_CAT_LABEL[cat]+'.';
  return { baseline:baseline, V:v, P:pPts, rawP:pRaw, proteinCounted:proteinCounts, F:fPts, score:score,
stars:stars, rows:rows, rule:rule, category:cat };
}

/* ---------- Codex claim engine (CAC/GL 23-1997; EU variation Reg. (EC) 1924/2006) ---------- */
function evaluateClaims(p, jurisdiction){
  var bev = (p.form === 'beverage');
  var unit = bev ? '100 ml' : '100 g';
  var kcal = p.energyKcal || 0;
  var out = [];
  function add(id, label, status, valueTxt, needTxt, note){
    out.push({ id:id, label:label, status:status, value:valueTxt, need:needTxt, note:note||'' });
  }
  function simple(id, label, value, limit, unitTxt){
    var pass = value <= limit + 1e-9;
    add(id, label, pass?'pass':'fail', fmt(value,2)+' '+unitTxt, '≤ '+fmt(limit,2)+' '+unitTxt+' per '+unit,
        pass?'':'needs to fall by '+fmt(value-limit,2)+' '+unitTxt);
  }
  /* Energy */
  simple('low-energy','Low energy', kcal, bev?20:40, 'kcal');
  if (bev) simple('energy-free','Energy-free', kcal, 4, 'kcal');
  else add('energy-free','Energy-free','na','—','Codex sets this claim per 100 ml (liquids)','not applicable to
```

```
solids in this tool');
  /* Fat */
  simple('low-fat','Low fat', p.fat, bev?1.5:3, 'g');
  simple('fat-free','Fat-free', p.fat, 0.5, 'g');
  var satPct = kcal>0 ? (p.satfat*9/kcal*100) : (p.satfat>0? Infinity : 0);
  var satLim = bev?0.75:1.5;
  var satPass = (p.satfat <= satLim+1e-9) && (satPct <= 10+1e-9);
  add('low-sat-fat','Low saturated fat', satPass?'pass':'fail',
      fmt(p.satfat,2)+' g and '+fmt(satPct,1)+'% of energy from saturated fat',
      '≤ '+fmt(satLim,2)+' g per '+unit+' AND saturated-fat energy ≤ 10% of total energy',
      satPass?'':(p.satfat>satLim?'saturated fat too high':'the 10%-of-energy sub-rule fails'));
  simple('sat-fat-free','Saturated-fat-free', p.satfat, 0.1, 'g');
  /* Sugars */
  simple('low-sugars','Low sugars', p.sugars, bev?2.5:5, 'g');
  simple('sugars-free','Sugars-free', p.sugars, 0.5, 'g');
  add('no-added-sugars','No added sugars', p.addedSugars?'fail':'pass',
      p.addedSugars?'added sugars present':'no added sugars declared',
      'no added mono-/disaccharides or other food used for its sweetening properties',
      (!p.addedSugars && p.sugars>0) ? 'pack must also state “contains naturally occurring sugars”' : '');
  /* Sodium */
  simple('low-sodium','Low sodium', p.sodiumMg, 120, 'mg');
  simple('very-low-sodium','Very low sodium', p.sodiumMg, 40, 'mg');
  simple('sodium-free','Sodium-free', p.sodiumMg, 5, 'mg');
  /* Protein */
  var per100kcalProtein = kcal>0 ? p.protein/kcal*100 : (p.protein>0? Infinity : 0);
  var protEnergyPct = kcal>0 ? p.protein*4/kcal*100 : 0;
  if (jurisdiction === 'eu'){
    add('source-protein','Source of protein (EU basis)', protEnergyPct>=12-1e-9?'pass':'fail',
        fmt(protEnergyPct,1)+'% of energy from protein', 'protein ≥ 12% of energy (Reg. (EC) 1924/2006)',
        'EU variation — replaces the Codex per-100 g / per-100 kcal routes');
    add('high-protein','High protein (EU basis)', protEnergyPct>=20-1e-9?'pass':'fail',
        fmt(protEnergyPct,1)+'% of energy from protein', 'protein ≥ 20% of energy (Reg. (EC) 1924/2006)',
        'EU variation — replaces the Codex per-100 g / per-100 kcal routes');
  } else {
    var srcLim = bev?2.5:5, highLim = bev?5:10;
    var srcRoute = (p.protein>=srcLim-1e-9) ? ('per '+unit+' route') : ((per100kcalProtein>=2.5-1e-9) ? 'per 100
kcal route' : '');
    add('source-protein','Source of protein', srcRoute?'pass':'fail',
        fmt(p.protein,2)+' g per '+unit+'; '+fmt(per100kcalProtein,2)+' g per 100 kcal',
        '≥ '+fmt(srcLim,1)+' g per '+unit+' (10% NRV) OR ≥ 2.5 g per 100 kcal (5% NRV)',
        srcRoute?('met via '+srcRoute):'neither route met');
    var highRoute = (p.protein>=highLim-1e-9) ? ('per '+unit+' route') : ((per100kcalProtein>=5-1e-9) ? 'per 100
kcal route' : '');
    add('high-protein','High protein', highRoute?'pass':'fail',
        fmt(p.protein,2)+' g per '+unit+'; '+fmt(per100kcalProtein,2)+' g per 100 kcal',
        '≥ '+fmt(highLim,1)+' g per '+unit+' (20% NRV) OR ≥ 5 g per 100 kcal (10% NRV)',
        highRoute?('met via '+highRoute):'neither route met');
  }
  /* Fibre */
  var per100kcalFibre = kcal>0 ? p.fibre/kcal*100 : (p.fibre>0? Infinity : 0);
  function fibreClaim(id, label, wLim, mlLim, kLim){
    var routes = [];
    if (!bev && p.fibre>=wLim-1e-9) routes.push('per 100 g route');
    if (bev && p.fibre>=mlLim-1e-9) routes.push('per 100 ml route');
    if (per100kcalFibre>=kLim-1e-9) routes.push('per 100 kcal route');
    add(id, label, routes.length?'pass':'fail',
        fmt(p.fibre,2)+' g per '+unit+'; '+fmt(per100kcalFibre,2)+' g per 100 kcal',
        (bev?('≥ '+fmt(mlLim,1)+' g per 100 ml'):('≥ '+fmt(wLim,1)+' g per 100 g'))+' OR ≥ '+fmt(kLim,1)+' g per
100 kcal',
        routes.length?('met via '+routes.join(' + ')):'no route met');
  }
  fibreClaim('source-fibre','Source of fibre', 3, 1.5, 1.5);
  fibreClaim('high-fibre','High fibre', 6, 3, 3);
  /* Vitamins & minerals (NRV routes per 100 g / 100 ml) */
  function micro(id, label, value, unitTxt, nrv){
    var srcLim = nrv*(bev?0.075:0.15), highLim = nrv*(bev?0.15:0.30);
    add('source-'+id, 'Source of '+label, value>=srcLim-1e-9?'pass':'fail',
        fmt(value,3)+' '+unitTxt+' per '+unit, '≥ '+fmt(srcLim,3)+' '+unitTxt+' per '+unit+' ('+(bev?'7.5':'15')
+'% of NRV '+fmt(nrv,0)+' '+unitTxt+')',
        value>=srcLim-1e-9?'':'needs ' + fmt(srcLim-value,3) + ' ' + unitTxt + ' more');
    add('high-'+id, 'High '+label, value>=highLim-1e-9?'pass':'fail',
        fmt(value,3)+' '+unitTxt+' per '+unit, '≥ '+fmt(highLim,3)+' '+unitTxt+' per '+unit+' ('+(bev?'15':'30')
+'% NRV)',
        value>=highLim-1e-9?'':'needs ' + fmt(highLim-value,3) + ' ' + unitTxt + ' more');
  }
  micro('calcium','calcium', p.calcium||0, 'mg', NRV.calcium);
  micro('iron','iron', p.iron||0, 'mg', NRV.iron);
```

```
  micro('vitc','vitamin C', p.vitC||0, 'mg', NRV.vitC);
  micro('vitd','vitamin D', p.vitD||0, 'µg', NRV.vitD);
  /* Caution flags (NOT Codex disqualifications) */
  var passCount = 0; out.forEach(function(c){ if (c.status==='pass') passCount++; });
  var flags = [];
  if (passCount>0){
    if (p.sodiumMg>=500) flags.push('sodium is '+fmt(p.sodiumMg,0)+' mg per '+unit+' (≥500)');
    if (!bev && p.sugars>=15) flags.push('sugars are '+fmt(p.sugars,1)+' g per 100 g (≥15)');
    if (p.satfat>=5) flags.push('saturated fat is '+fmt(p.satfat,1)+' g per '+unit+' (≥5)');
    if (kcal>=400) flags.push('energy is '+fmt(kcal,0)+' kcal per '+unit+' (≥400)');
  }
  return { claims: out, passCount: passCount, flags: flags };
}

/* ---------- Batch CSV parser ---------- */
var BATCH_COLS =
['name','form','energy_kcal','protein','fibre','sugars','fat','satfat','sodium_mg','fvn_pct','calcium','iron','v
itc','vitd'];
function normKey(s){ return (''+s).toLowerCase().replace(/[\s\-]+/g,'_').replace(/[^a-z0-9_]/g,''); }
function splitLine(line){
  var delim = line.indexOf(';')>=0 ? ';' : (line.indexOf('\t')>=0 ? '\t' : ',');
  return line.split(delim).map(function(x){ return x.trim().replace(/^"|"$/g,''); });
}
function parseBatchCSV(text){
  var lines = (''+text).split(/\r?\n/); var products = [], errors = [];
  var colMap = null, firstData = 0;
  for (var i=0;i<lines.length;i++){
    if (lines[i].trim()===''){ continue; }
    var cells = splitLine(lines[i]);
    var keys = cells.map(normKey);
    var isHeader = keys.indexOf('name')>=0 || keys.indexOf('energy_kcal')>=0 || keys.indexOf('protein')>=0;
    if (isHeader && colMap===null){
      colMap = {};
      keys.forEach(function(k, idx){ if (BATCH_COLS.indexOf(k)>=0) colMap[k]=idx; });
      if (colMap.name===undefined) colMap.name = 0;
      if (colMap.energy_kcal===undefined && keys.indexOf('energy')>=0) colMap.energy_kcal =
keys.indexOf('energy');
      firstData = i+1; break;
    }
    break;
  }
  var start = (colMap!==null) ? firstData : 0;
  if (colMap===null){ colMap = {}; BATCH_COLS.forEach(function(k, idx){ colMap[k]=idx; }); }
  for (var j=start;j<lines.length;j++){
    var raw = lines[j]; if (raw.trim()==='') continue;
    var c = splitLine(raw); var lineNo = j+1;
    function cell(k){ var ix = colMap[k]; return (ix===undefined || ix>=c.length) ? '' : c[ix]; }
    function num(k){ var v = cell(k); if (v===''||v===undefined) return 0; var n =
parseFloat((''+v).replace(',','.')); return isNaN(n)? null : n; }
    var name = cell('name') || ('Product '+(products.length+1));
    var formRaw = (cell('form')||'solid').toLowerCase();
    var form = (formRaw.indexOf('bev')>=0 || formRaw.indexOf('drink')>=0 || formRaw.indexOf('liquid')>=0) ?
'beverage' : 'solid';
    var kcal = num('energy_kcal'), protein = num('protein'), fibre = num('fibre'), sugars = num('sugars'),
        fat = num('fat'), satfat = num('satfat'), sodium = num('sodium_mg'), fvn = num('fvn_pct');
    if ([kcal,protein,fibre,sugars,fat,satfat,sodium,fvn].indexOf(null)>=0){
      errors.push('Line '+lineNo+': non-numeric value in a nutrient column — row skipped.');
      continue;
    }
    products.push({
      name: name, form: form, serving: 0, fvn: Math.min(100, Math.max(0, fvn)), fvnConcentrated: false,
      redMeat: false, addedSugars: false, sweeteners: false, water: false, hsrCategory:
form==='beverage'?'1':'2',
      energyKcal: kcal, energyKj: kcal*KJ_PER_KCAL, protein: protein, fibre: fibre, sugars: sugars,
      fat: fat, satfat: satfat, sodiumMg: sodium, saltG: sodiumMgFromSaltG(0)+saltGFromSodiumMg(sodium),
      calcium: num('calcium')||0, iron: num('iron')||0, vitC: num('vitc')||0, vitD: num('vitd')||0
    });
  }
  return { products: products, errors: errors };
}

function evaluateAll(p, jurisdiction){
  return { claims: evaluateClaims(p, jurisdiction||'codex'), nutri: nutriScore(p), hsr: healthStars(p) };
}
/*PURE-CORE-END*/
```